

CHARACTERS AND TEXT

From a single letter to a full string

THE NEED

Category codes

```
char foodCode{'F'};  
char transportCode{'T'};  
char entertainmentCode{'E'};
```

Our expense tracker sorts spending into categories, and a single letter is a handy, compact code for one.

WHAT A CHAR REALLY IS

One byte, mapped by ASCII

```
std::println("sizeof(char): {}", sizeof(char));
```

A **char** is just one byte in memory: $2^8 = 256$ possible values (0-255), each one mapped to a character by the ASCII table.

CHAR AS A SMALL INTEGER

Numbers underneath the letters

```
char fromCode{70}; // ASCII code 70 is 'F'  
std::println("fromCode (as int): {}", static_cast<int>(fromCode));
```

Because a `char` is a small integer under the hood, we can store its numeric code directly and `static_cast` it back to see the number.

CHAR ARITHMETIC

Stepping through codes

```
char nextCategoryCode = static_cast<char>(entertainmentCode + 1);  
// 'F' - the next letter after 'E'
```

Chars behave like small integers for arithmetic too - handy for stepping through category codes when adding a new one.

BEYOND ONE LETTER

std::string for full text

```
std::string expenseDescription{"Coffee with client"};
```

A single **char** can only ever hold one letter. For a full expense description we reach for **std::string** instead - the same type already used for full names since chapter 3.

DEBUG IT

See the number behind the letter

Breakpoint after `fromCode` is initialized - most debuggers show a `char` as both its letter and its numeric ASCII value in the same watch entry, which is the fastest way to build intuition for the mapping.